Rekurzia

Ján Mazák

FMFI UK Bratislava

Rekurzia: cez CTE (Common Table Expressions)

WITH RECURSIVE umožňuje rekurzívne sa odvolať na reláciu.

```
WITH RECURSIVE cte_name AS(
    SELECT ...  -- non-recursive term
  UNION [ALL]
    SELECT ...  -- recursive term
)
SELECT * FROM cte_name;
```

Rekurzia

```
WITH RECURSIVE t(n) AS (
    SELECT 1
  UNION ALL
    SELECT n+1 FROM t WHERE n < 5
)
SELECT n FROM t;
```

Výpočet rekurzie — iterácia (seminaive evaluation)

Naivná evaluácia v každom kroku znova vyhodnotí rekurzívny člen nad všetkými doterajšími riadkami; seminaivná spracúva len riadky pridané v predchádzajúcom kroku.

1. Evaluate the non-recursive term. For UNION (but not UNION ALL), discard duplicate rows. Include all remaining rows in the result of the recursive query, and also place them in a temporary working table.
2. Evaluate the recursive term, substituting the current contents of the working table for the recursive self-reference. For UNION, discard duplicate rows and rows that duplicate any previous result row. Include all remaining rows in the result of the recursive query, and also place them in a temporary intermediate table.
3. If the intermediate table is empty, stop. Otherwise replace the working table with the intermediate table and go to 2.

Rekurzia

Rekurzívny odkaz možno použiť len raz a pre jednu reláciu (PostgreSQL). Nekorektné využitie:

```
WITH RECURSIVE t(n) AS (
    SELECT 1
  UNION ALL
    (SELECT n+1 FROM t WHERE n < 5
        UNION ALL
     SELECT n-1 FROM t WHERE n < 5)
)
SELECT n FROM t;
```

Rekurzia

Rekurzívny výpočet nemusí skončiť, zodpovednosť je na autorovi dotazu.

```
WITH RECURSIVE t(n) AS (
    SELECT 1
  UNION ALL
    SELECT n+1 FROM t
)
SELECT n FROM t WHERE n < 5;
```

(v PostgreSQL sa takto dotaz nezastaví)

Rekurzia

Rekurzia zvyšuje vyjadrovaciu silu jazyka SQL.

- Rekurziou sa dá vyjadriť tranzitívny uzáver, t. j. existencia cesty neobmedzenej dĺžky.
- Bez rekurzie možno vyjadriť len cestu fixnej dĺžky (pomocou opakovaného joinu).

Prehľadávanie grafov

Pomocou rekurzie možno zapísať prehľadávanie stromu. Bez určenia počiatočného vrcholu ide o prehľadávanie zo všetkých vrcholov naraz; počiatočný vrchol určíme podmienkou v prvej časti (WHERE t.vFrom = ...).

```
WITH RECURSIVE search_tree(vFrom, vTo) AS (
    SELECT t.vFrom, t.vTo
    FROM tree t
  UNION ALL
    SELECT st.vFrom, t.vTo
    FROM search_tree st, tree t
    WHERE t.vFrom = st.vTo
)
SELECT * FROM search_tree;
```

Prehľadávanie grafov

Poradie riadkov vo výsledku nie je zaručené, ale usporiadaním podľa hĺbky dostaneme vrcholy v poradí prehľadávania do šírky.

```
WITH RECURSIVE search_tree(vFrom, vTo, depth) AS (
    SELECT t.vFrom, t.vTo, 1
    FROM tree t
  UNION ALL
    SELECT st.vFrom, t.vTo, depth + 1
    FROM search_tree st, tree t
    WHERE t.vFrom = st.vTo
)
SELECT * FROM search_tree ORDER BY depth;
```

Prehľadávanie grafov

Poradie riadkov vo výsledku nie je zaručené, ale usporiadaním podľa cesty dostaneme vrcholy v poradí prehľadávania do hĺbky.

```
WITH RECURSIVE search_tree(vFrom, vTo, path) AS (
    SELECT t.vFrom, t.vTo, ARRAY[t.vFrom, t.vTo]
    FROM tree t
  UNION ALL
    SELECT st.vFrom, t.vTo, path || t.vTo
    FROM search_tree st, tree t
    WHERE t.vFrom = st.vTo
)
SELECT * FROM search_tree ORDER BY path;
```

Prehľadávanie grafov

Ak chceme ustrážiť koniec rekurzie, treba sa vyhýbať zacykleniu.

```
WITH RECURSIVE
    search_graph(vFrom, vTo, depth, path, is_cycle)
AS (
    SELECT g.vFrom, g.vTo, 1,
        ARRAY[g.vFrom, g.vTo], false
    FROM graph g
  UNION ALL
    SELECT sg.vFrom, g.vTo, sg.depth + 1,
        path || g.vTo, g.vTo = ANY(path)
    FROM search_graph sg, graph g
    WHERE g.vFrom = sg.vTo AND NOT is_cycle
)
SELECT * FROM search_graph ORDER BY is_cycle, path;
```

Prehľadávanie grafov

SQL po pridaní rekurzie je turingovsky úplný jazyk (vyjadríte v ňom riešenie pre všetky veci programovateľné v Pythone či C). Náčrt dôkazu a ukážka riešenia zložitých problémov v SQL, napr. ASCII Mandelbrotovej množiny: link

Prehľadávanie grafov

SQL po pridaní rekurzie je turingovsky úplný jazyk (vyjadríte v ňom riešenie pre všetky veci programovateľné v Pythone či C). Náčrt dôkazu a ukážka riešenia zložitých problémov v SQL, napr. ASCII Mandelbrotovej množiny: link

Neznamená to však, že túto silu musíte aj využiť. Napr. pri prehľadávaní grafu treba zvážiť, či nie je jednoduchšie vypýtať si celý graf z databázy a prehľadávanie zapísať ako program vo vhodnejšom jazyku. Podobne pri iných komplikovaných problémoch (napr. riešených backtrackingom).

Hierarchické dáta sa však v databázach občas vyskytujú, treba preto mať aspoň predstavu, ako sa s nimi vysporiadať.

Literatúra

- https://www.postgresql.org/docs/current/queries-with.html
- https://www.sqlite.org/lang_with.html

Na zamyslenie

- Dá sa každá rekurzia zapísať pomocou iterácie?
  A čo iterácia pomocou rekurzie?
- Vysvetlite rozdiel medzi naivnou a seminaivnou evaluáciou rekurzívneho dotazu.
- Vymyslite otázku, ktorá sa nedá vyjadriť v SQL ani po pridaní rekurzie. (Bežne sa pýtame na záznamy, to sa vyjadriť dá, ale skúste sa pýtať na niečo iné.)

Úlohy

Čo počíta tento dotaz?

```
WITH RECURSIVE t(s) AS (
    SELECT 'foo'
  UNION ALL
    SELECT s || 'bar' FROM t WHERE length(s) < 20
)
SELECT s FROM t ORDER BY s;
```

Úlohy

Čo počíta tento dotaz?

```
WITH RECURSIVE t(s) AS (
    SELECT 'foo'
  UNION ALL
    SELECT s || 'bar' FROM t WHERE length(s) < 20
)
SELECT s FROM t ORDER BY s;
```

Generuje reťazce v tvare `foo(bar)`* postupne od najkratších.

Úlohy

Súčet čísel od 1 do 100

Úlohy

Súčet čísel od 1 do 100

```
WITH RECURSIVE t(n) AS (
    VALUES (1)
  UNION ALL
    SELECT n+1 FROM t WHERE n < 100
)
SELECT sum(n) FROM t;
```

Úlohy

Čo počíta tento dotaz?
Zmení sa to, ak UNION nahradíme UNION ALL?

```
WITH RECURSIVE t(n) AS (
    SELECT 1
  UNION
    SELECT 10-n FROM t
)
SELECT * FROM t ORDER BY n;
```

Úlohy

Skončí výpočet tohto dotazu?

```
WITH RECURSIVE t(n) AS (
    VALUES (1)
  UNION ALL
    SELECT n+1 FROM t
)
SELECT * FROM t LIMIT 10;
```

Úlohy

Skončí výpočet tohto dotazu?

```
WITH RECURSIVE t(n) AS (
    VALUES (1)
  UNION ALL
    SELECT n+1 FROM t
)
SELECT * FROM t LIMIT 10;
```

V PostgreSQL áno, ale vo všeobecnosti to závisí od implementácie LIMIT a rekurzie.